LEMBAR KERJA PRAKTIKUM 7 STRUKTUR DATA

TABEL HASH & HASHING

A. Tujuan Praktikum

1. Mengimplementasikan hash table dalam bahasa C.
2. Memahami konsep hash function dan collision.
3. Mengimplementasikan collision handling menggunakan chaining dan linear probing.
4. Mengamati pengaruh collision terhadap performa operasi hash table.
5. Memahami konsep load factor dan rehashing.

B. Implementasikan Program di bawah ini

Program 1. Program Hash Table (Separate Chaining)

```
#include <iostream>
using namespace std;

#define TABLE_SIZE 7

struct Node{
    int key;
    Node* next;
};

Node* hashTable[TABLE_SIZE];

int hashFunction(int key){
    return key % TABLE_SIZE;
}

Node* createNode(int key){
    Node* newNode = new Node();
    newNode->key = key;
    newNode->next = NULL;
    return newNode;
}

void insert(int key){

    int index = hashFunction(key);

    Node* newNode = createNode(key);

    if(hashTable[index] == NULL){
        hashTable[index] = newNode;
    }
    else{
```

```
        newNode->next = hashTable[index];
        hashTable[index] = newNode;
    }
}

void display(){

    for(int i=0;i<TABLE_SIZE;i++){

        cout << i << " -> ";

        Node* temp = hashTable[i];

        while(temp != NULL){
            cout << temp->key << " -> ";
            temp = temp->next;
        }

        cout << "NULL" << endl;
    }
}

int main(){

    for(int i=0;i<TABLE_SIZE;i++)
        hashTable[i] = NULL;

    int n,x;

    cout << "Masukkan jumlah data: ";
    cin >> n;

    cout << "Masukkan data:" << endl;

    for(int i=0;i<n;i++){
        cin >> x;
        insert(x);
    }

    cout << "\nHash Table:" << endl;
    display();

    return 0;
}
```

Program 2. Program Hash Table (Linear Probing)

```
#include <iostream>
using namespace std;
```

```
#define TABLE_SIZE 7
#define EMPTY -1

int hashTable[TABLE_SIZE];

int hashFunction(int key){
    return key % TABLE_SIZE;
}

void insert(int key){

    int index = hashFunction(key);

    while(hashTable[index] != EMPTY){
        index = (index + 1) % TABLE_SIZE;
    }

    hashTable[index] = key;
}

void display(){

    for(int i = 0; i < TABLE_SIZE; i++){

        if(hashTable[i] == EMPTY)
            cout << i << " -> kosong" << endl;
        else
            cout << i << " -> " << hashTable[i] << endl;
    }
}

int main(){

    for(int i = 0; i < TABLE_SIZE; i++)
        hashTable[i] = EMPTY;

    int n, x;

    cout << "Masukkan jumlah data: ";
    cin >> n;

    cout << "Masukkan data:" << endl;

    for(int i = 0; i < n; i++){
        cin >> x;
        insert(x);
    }

    cout << "\nHash Table:" << endl;
    display();

    return 0;
```

}

Setelah mengimplementasikan kedua program di atas, lakukan pengujian menggunakan empat jenis data berikut.

Data Acak
Gunakan data berikut:
15 11 27 8 12 7 18
Lakukan:

1. Jalankan **Program 1 (Chaining)**
2. Jalankan **Program 2 (Linear Probing)**

Amati:

- posisi penyimpanan data dalam tabel
- jumlah collision yang terjadi
- bagaimana collision ditangani oleh masing-masing metode

Data dengan Pola Kelipatan
Gunakan data berikut:
7 14 21 28 35 42
Amati:

- bagaimana distribusi data pada chaining
- bagaimana proses probing pada linear probing
- apakah terbentuk **cluster** pada tabel

Data dengan Hash Sama
Gunakan data berikut:
10 17 24 31 38
Semua data memiliki nilai:
k mod 7 = 3
Sehingga semua key memiliki **hash yang sama**.
Amati:

1. bagaimana chaining menyimpan semua data pada slot yang sama
2. bagaimana linear probing mencari slot kosong
3. apakah terbentuk **primary clustering**

Data Banyak Elemen (Load Factor Tinggi)
Gunakan data berikut:
5 12 14 20 33 45 56
Jumlah data hampir sama dengan ukuran tabel.
Amati:

- apakah collision semakin sering terjadi
- bagaimana performa pencarian

- apakah distribusi data masih merata
- Jika jumlah data terus bertambah, apa yang seharusnya dilakukan?

C. Berdasarkan program yang telah Anda jalankan, lakukan proses tracing dan gunakan fitur watch variables untuk mengamati alur kerja program secara detail. Setelah memahami prosesnya, diskusikan pertanyaan-pertanyaan berikut bersama asisten dan teman-teman Anda.

1. Apa perbedaan cara penyimpanan data antara Separate Chaining dan Linear Probing ketika terjadi collision pada hash table?
2. Berdasarkan hasil pengujian dengan berbagai data, bagaimana distribusi data pada tabel untuk kedua metode tersebut? Apakah distribusinya terlihat merata?
3. Pada saat terjadi collision, bagaimana masing-masing metode menangani masalah tersebut, dan metode mana yang terlihat lebih sederhana untuk dipahami dari hasil program?
4. Ketika menggunakan data dengan pola kelipatan (misalnya 7, 14, 21, dan seterusnya), bagaimana pengaruh pola data tersebut terhadap hasil fungsi hash k mod 7?
5. Apakah pola data tertentu dapat menyebabkan distribusi hash menjadi tidak merata? Jelaskan berdasarkan hasil pengamatan dari program.
6. Ketika semua data memiliki nilai hash yang sama, bagaimana perbedaan cara penyimpanan data antara metode chaining dan linear probing?
7. Pada metode linear probing, apakah terlihat terbentuknya cluster (sekumpulan slot terisi yang berdekatan)? Bagaimana cluster tersebut dapat mempengaruhi proses pencarian atau penyisipan data?
8. Ketika jumlah data mendekati ukuran tabel (load factor tinggi), apa yang terjadi terhadap jumlah collision dan proses penyisipan data pada kedua metode?
9. Dari hasil percobaan, metode mana yang menurut Anda lebih stabil ketika jumlah collision meningkat? Jelaskan alasan Anda berdasarkan hasil yang diamati.
10. Jika jumlah data terus bertambah melebihi kapasitas tabel, solusi apa yang seharusnya dilakukan dalam implementasi hash table agar performanya tetap efisien?

D. Video dan Hasil Diskusi (dikumpulkan selambatnya satu minggu setelah praktikum ini)

Setelah Anda mempelajari program di atas, kerjakan tugas di bawah ini dengan kelompok Anda, lalu buat video diskusinya dan tuliskan hasil diskusinya (PDF), masukkan ke gdrive, lalu kumpulkan link gdrive pada form berikut: https://forms.gle/JCWa8XJy9n7Vnc7r7.

Pada program sebelumnya terdapat deklarasi ukuran tabel sebagai berikut:

```
#define TABLE_SIZE 7
```

Nilai tersebut menunjukkan bahwa hash table memiliki 7 slot indeks (0–6).

Ubahlah nilai TABLE_SIZE menjadi beberapa ukuran yang berbeda.

Contoh:

#define TABLE_SIZE 10

atau

#define TABLE_SIZE 15

Setelah mengubah ukuran tabel, jalankan kembali program menggunakan data input yang sama.

Gunakan data berikut:

50
700
76
85
92
73
101

Amati perubahan posisi penyimpanan data pada hash table dan perhatikan apakah terjadi collision.

Langkah Percobaan

1. Jalankan program dengan ukuran tabel: TABLE_SIZE = 7
2. Catat hasil hash table yang dihasilkan pada tabel pengamatan.
3. Ubah ukuran tabel menjadi: TABLE_SIZE = 10
4. Jalankan kembali program dengan data yang sama.
5. Catat kembali hasil hash table.
6. Ubah ukuran tabel menjadi: TABLE_SIZE = 15
7. Jalankan kembali program dan amati hasilnya.

Isikan ke dalam Tabel Pengamatan

Ukuran Tabel	Jumlah Data	Jumlah Collision	Keterangan
7			
10			
15			

Pertanyaan diskusi:

1. Apa yang terjadi pada jumlah collision ketika ukuran tabel diperbesar?
2. Mengapa ukuran hash table dapat mempengaruhi jumlah collision?
3. Bandingkan hasil percobaan pada ukuran tabel 7, 10, dan 15. Pada ukuran tabel mana collision paling sedikit?
4. Jelaskan hubungan antara load factor dengan kemungkinan terjadinya collision.
5. Menurut Anda, apa dampak collision terhadap performa operasi pencarian (search) pada hash table?